

Informe de Cumplimiento

Práctica Experimental Unidad 5: CI/CD, Pruebas y Observabilidad

Sistema de Gestión de Solicitudes de Soporte Técnico de Internet

Repositorio: PFC-SOPORTE-ISP

Carlos José Carpio Mendoza

Cristhian Daniel Pacheco Cárdenas

Robinson Rodrigo Cando Moreno

Jeremy Álvarez

Equipo ACC

Facultad de Ciencias de la Computación

Universidad Técnica Estatal de Quevedo

Agosto 2026

Resumen

Este informe verifica, con evidencia directa del código fuente y la configuración del repositorio `carlospatroner-boop/PFC-SOPORTE-ISP`, el grado de cumplimiento de la Parte A de la Práctica Experimental de la Unidad 5: un pipeline CI/CD con pruebas automáticas, publicación de imágenes Docker y análisis estático; y observabilidad básica con logging estructurado, métricas de negocio en formato Prometheus y un dashboard de Grafana. La verificación se realizó clonando la rama `feature/entrega-4` (la que contiene el trabajo más reciente) e inspeccionando directamente los archivos de workflow, configuración y código fuente, sin asumir cumplimiento a partir de nombres de commit o comentarios.

1. Introducción

El equipo ACC desarrolla *PFC-SOPORTE-ISP*, un sistema distribuido de gestión de tickets de soporte técnico para un ISP, compuesto por seis microservicios (`auth-service`, `svc-principal/ticket-service`, `report-service`, `api-gateway`, `notification-service` y `ai-service`), un clúster CockroachDB, mensajería Kafka y aplicaciones web y móvil. La actividad culminante del período (Práctica Experimental Unidad 5) exige que el equipo demuestre, sobre este mismo proyecto, dos capacidades de ingeniería de software modernas: integración/entrega continua (CI/CD) y observabilidad. Este documento contrasta punto por punto los requisitos de la Parte A del enunciado con lo que realmente existe en el repositorio.

2. Metodología de verificación

Se clonó el repositorio público y se examinó tanto la rama `main` como `feature/entrega-4`. La rama `main` solo contiene el workflow simple de la Entrega 3 (`ci.yml`, sólo pruebas unitarias). Todo el trabajo relevante para esta práctica —pipeline CI/CD completo, métricas, logging estructurado y dashboard— vive en `feature/entrega-4`, que es la rama evaluada en este informe. Para cada criterio del enunciado se citó el archivo exacto que lo sustenta (workflow de GitHub Actions, `Dockerfile`, `application.yml`, `logback-spring.xml`, `docker-compose.yml`, `prometheus.yml` y `pfc-dashboard.json`),

evitando inferir cumplimiento a partir de mensajes de commit o comentarios de código que no estén respaldados por configuración real. Esta práctica de auditoría dirigida por evidencia responde a la recomendación empírica de que la adopción de CI/CD debe medirse sobre artefactos verificables del repositorio y no solo sobre su presencia declarada [1].

3. Parte A — Pipeline CI/CD

El pipeline vive en `.github/workflows/ci-cd.yml` (rama `feature/entrega-4`) y se dispara en cada `push` a `main` y a cualquier rama `feature/**`, y en cada `pull_request` hacia `main`.

3.1. (a) Pruebas unitarias automáticas en cada push

Cumple. Tres jobs independientes ejecutan la suite de pruebas completa en cada disparo del workflow: `test-backend` corre `mvn test` para los tres servicios Java con Maven (`auth-service`, `svc-principal`, `report-service`) y además `npm test` para `notification-service` y `pytest` para `ai-service`; `test-web` corre las pruebas de `apps/web` con Vitest; `test-mobile` corre pruebas unitarias de ViewModel y pruebas instrumentadas de Compose en un emulador Android headless. Ningún job depende de ejecución manual.

3.2. (b) Build y publicación de la imagen Docker en GHCR

Cumple. El job `build-images` usa una matriz de 7 imágenes (los 6 microservicios más `apps/web`), cada una con su propio `Dockerfile`, y las publica en `ghcr.io` mediante `docker/login-action@v3` y `docker/build-push-action@v5`, con tags por `sha` de commit y `latest`. Corre solo en eventos `push` (no en `pull_request`, para no llenar el registro con imágenes de prueba) y depende de que `test-backend` y `test-web` hayan pasado primero.

3.3. (c) Análisis estático de código

Cumple parcialmente. El job `lint` ejecuta ESLint (`npm run lint`) sobre `apps/web` y Android Lint (`gradlew lintDebug`) sobre `apps/mobile`, ambos como pasos obligatorios del pipeline —esto satisface literalmente la opción “Pylint/ESLint configurado en el pipeline” del enunciado. Sin embargo, la cobertura no es uniforme: no se encontró configuración de SonarCloud (sin `sonar-project.properties` ni acción `sonarcloud-github-action`), no hay Pylint ni Flake8 para `ai-service` (Python), y ninguno de los cuatro servicios Java del backend (`auth-service`, `svc-principal`, `report-service`, `api-gateway`) tiene un analizador estático (Checkstyle, PMD o SpotBugs) en el pipeline. Yeboah y Popoola muestran empíricamente que la herramienta elegida importa: SonarQube dominó a PMD, Checkstyle y FindBugs en precisión y recall sobre Java, C++ y Python [2], lo que hace de SonarCloud (su equivalente en la nube, gratuito para repositorios públicos) la opción más costo-efectiva para cerrar esta brecha sin multiplicar herramientas por lenguaje.

4. Parte A — Observabilidad básica

4.1. (a) Logging estructurado (JSON) en todos los microservicios

Cumple parcialmente (4 de 6 servicios). `auth-service`, `svc-principal`, `report-service` y `api-gateway` configuran Logback con `LogstashEncoder` (`logback-spring.xml`), que emite cada línea de log como un objeto JSON e incluye `trace_id` y `span_id` vía MDC más un campo `service` fijo por microservicio. En cambio, `notification-service` (Node/Express) registra con

`console.log/console.error` de texto plano sin estructura ni correlación, y `ai-service` (Python/FastAPI) usa `logging.basicConfig` con un formato de texto `%(asctime)s%(levelname)s...`, también sin JSON. El enunciado pide explícitamente JSON en *todos* los microservicios, por lo que esta es la brecha más clara del proyecto en la parte de observabilidad. Esta brecha no es solo formal: conjuntos de datos reales de producción muestran que los logs y las métricas se analizan cada vez más de forma conjunta (fusión multimodal) para detectar anomalías en microservicios [3], algo que exige que el log de *todos* los servicios — no solo los Java — tenga una estructura parseable de forma consistente.

4.2. (b) Al menos 3 métricas de negocio en `/metrics` (formato Prometheus)

Cumple y excede el mínimo. Los servicios Java exponen Spring Boot Actuator en `/actuator/prometheus` (`management.endpoints.web.exposure.include: health,info,prometheus`). Sobre esa base existen métricas de negocio hechas a medida y no solo las genéricas de la JVM: `HttpMetricsFilter/HttpMetricsGlobal` (tasa y latencia de peticiones por ruta), `CrdbMetrics` con `RepositoryTimingAspect` (reintentos de transacción y tiempos de repositorio contra CockroachDB) y `EscalationMetricsObserver` (escalamientos de tickets, una métrica de negocio del dominio de soporte). `ops/prometheus/prometheus.yml` scrapea las cuatro rutas Java cada 10 s, además del clúster CockroachDB y `cAdvisor`. Estas métricas se alinean con el enfoque de instrumentación liviana y específica del dominio que recomienda la literatura reciente sobre selección de métricas críticas en microservicios [4].

4.3. (c) Dashboard básico en Grafana con las 3 métricas visualizadas

Cumple funcionalmente, con una desviación de plataforma. `ops/grafana/pfc-dashboard.json` define un dashboard (“PFC Soporte ISP — Observabilidad”) con **6 paneles**, más del triple del mínimo exigido: tasa de peticiones, latencia p50/p95/p99, tasa de errores 5xx, disponibilidad Raft del clúster CockroachDB, uso de recursos por contenedor y latencia end-to-end en el API Gateway. Se aprovisiona automáticamente (`ops/grafana/provisioning`) al levantar `docker-compose.yml`. La desviación respecto al enunciado es de *plataforma*, no de contenido: el enunciado pide **Grafana Cloud (capa gratuita)**, un servicio alojado, mientras que la implementación actual es una instancia de Grafana *autoalojada* dentro del propio `docker-compose` del proyecto. Diseñar observabilidad evaluable exige decisiones explícitas sobre dónde y cómo se visualizan las señales [5]; en este caso el equipo priorizó la integración con el resto del stack local (Prometheus, Tempo, OTel Collector) sobre el uso literal de Grafana Cloud.

5. Tabla resumen de cumplimiento

Item	Requisito	Estado	Evidencia principal
CI/CD (a)	Pruebas unitarias en cada push	Cumple	ci-cd.yml: jobs test-backend, test-web, test-mobile
CI/CD (b)	Build y publicación de imagen Docker en GHCR	Cumple	ci-cd.yml: job build-images (7 imágenes)
CI/CD (c)	Análisis estático configurado	Parcial	ESLint y Android Lint sí; falta Sonar- Cloud/Pylint/Checkstyle en 5 de 7 componentes
Obs. (a)	Logging JSON en todos los microservicios	Parcial	4/6 con Logback+JSON; Node y Python en texto plano
Obs. (b)	≥ 3 métricas de negocio en /metrics	Cumple (excede)	CrdbMetrics, EscalationMetricsObserver, HttpMetricsFilter
Obs. (c)	Dashboard Grafana con las métricas	Cumple*	pfc-dashboard.json (6 paneles); *self-hosted, no Grafana Cloud

Tabla 1: Cumplimiento de la Parte A por criterio, rama `feature/entrega-4`.

6. Brechas identificadas y recomendaciones

1. **Análisis estático incompleto.** Incorporar SonarCloud (capa gratuita para repositorios públicos) como paso adicional del job `lint`, apuntando a los cuatro módulos Maven y a `ai-service`; alternativamente, añadir Pylint/Flake8 para `ai-service` y Checkstyle o SpotBugs para los módulos Java, de modo que los 7 componentes del monorepo queden cubiertos y no solo `apps/web` y `apps/mobile`.
2. **Logging no uniforme.** Migrar `notification-service` a un logger JSON (por ejemplo `pino`) y `ai-service` a `python-json-logger` o `structlog`, replicando el patrón ya usado en los servicios Java: un campo `service` fijo y correlación `trace_id/span_id` propagada desde el API Gateway.
3. **Grafana self-hosted vs. Grafana Cloud.** Si la rúbrica exige literalmente Grafana Cloud, importar `pfc-dashboard.json` tal cual a una cuenta gratuita de Grafana Cloud (el JSON ya es compatible) y apuntar su datasource Prometheus remoto al `remote_write` del Prometheus local; si la equivalencia funcional es aceptable, documentar explícitamente esta decisión de diseño en la entrega para evitar pérdida de puntos por una lectura literal del enunciado.
4. **Rama no fusionada.** Todo lo verificado en este informe vive en `feature/entrega-4`; `main` aún solo tiene el workflow simple de la Entrega 3. Fusionar la rama antes de la fecha de entrega, o confirmar con el docente que evaluará explícitamente `feature/entrega-4` o su Pull Request abierto.

7. Conclusión

De los seis criterios de la Parte A, tres se cumplen sin reservas (pruebas automáticas, publicación de imágenes en GHCR y métricas de negocio en Prometheus, esta última superando el mínimo exigido) y tres se cumplen parcialmente por brechas puntuales y acotadas: cobertura desapareja de análisis estático, logging no estructurado en dos de seis microservicios, y uso de Grafana self-hosted en lugar de Grafana Cloud. Ninguna brecha es estructural: las cuatro recomendaciones de la sección anterior son incrementales sobre la arquitectura de observabilidad y CI/CD ya construida, que en varios aspectos (número de paneles del dashboard, métricas de negocio expuestas) excede lo mínimo pedido por el enunciado.

Referencias

- [1] J. Fairbanks, A. Tharigonda, and N. U. Eisty, “Analyzing the effects of CI/CD on open source repositories in GitHub and GitLab,” 2023.
- [2] J. Yeboah and S. Popoola, “Efficacy of static analysis tools for software defect detection on open-source projects,” 2024.
- [3] A. Bakhtin, J. Nyssölä, Y. Wang, N. Ahmad, K. Ping, M. Esposito, M. Mäntylä, and D. Taibi, “LO2: Microservice API anomaly dataset of logs and metrics,” 2025.
- [4] A. Singal, D. Pathak, K. Ray, F. George, M. Verma, and P. Moogi, “Metric criticality identification for cloud microservices,” 2025.
- [5] M. C. Borges, J. Bauer, S. Werner, M. Gebauer, and S. Tai, “Informed and assessable observability design decisions in cloud-native microservice applications,” in *2024 IEEE 21st International Conference on Software Architecture (ICSA)*, 2024.